SubwayHorse's Design Doc for P04: Makers Makin' It, Act II -- The Seequel

Roster: Steven Wu, Alexandru Cimpoiesu, Jiefeng Ou, Cindy Liu
PROJECT NAME: CityCash
TARGET SHIP DATE: 2026-04-24

Summary

Our project aims to show the job prospects of different NYC Government employees. The user will be presented with a map of the city, color-coded by salary. The user will be able to compare salaries based on location, department, and job title by looking at different graphs. They can also look at the evolution of salaries over a specific time period. Our goal is to make it easy to compare job salaries, and judge job prospects based on certain factors.

Program Components

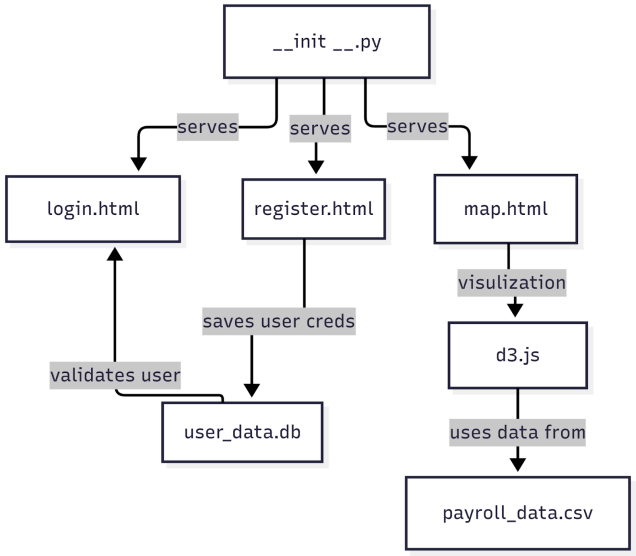
Frontend:

- HTML (frontend display)
  - login.html
  - register.html
  - map.html: Contains a map of NYC. There will be a search menu where you can find data you want (like job X's avg. salaries) and plot it against other data. Users can dynamically create graphs, or can compare stats across boroughs.
- CSS
  - Tailwind → style **login.html**, **register.html**, and **map.html**
- Javascript (data visualization)
  - d3.js → responsible for the map in **map.html**

Backend:

- sqlite3 (data storage system)
  - payroll_data → accessed by **map.html** to display data
  - user_data → accessed by **register.html** and **login.html** to facilitate user accounts
- Python (application layer)
  - __init__.py
- Flask (web server/delivery framework)

Component Map



Database Organization

We are using SQLite3, a relational database, for USER_DATA, because that's the way it's been done with proven success.

We are using a CSV file for PAYROLL_DATA because at a glance, it looks like d3.js uses CSV files to go through the data for their visualizations. Therefore, it'd be simpler to keep the data as CSV rather than putting it in a SQLite database and learning how to work it with js.

We will keep all the data from the dataset, because we believe that everything here is of use to someone (examples close to heart include looking for leave status of Mr. Moran, crazy OT hours, corruption or shenanigans in total other pay, etc, etc)

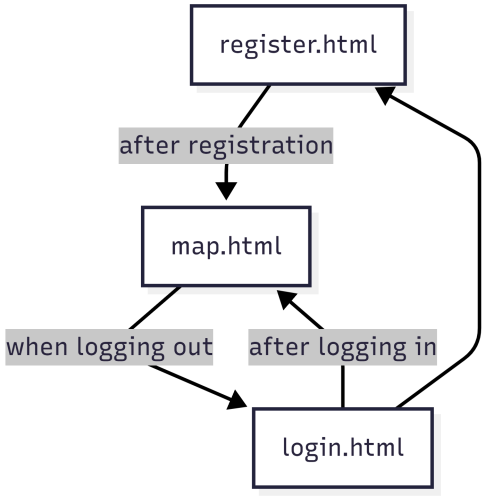
DB Schema

USER_DATA		
username	string	PK
password	string	

CSV Data

PAYROLL_DATA	
fiscal_year	int
payroll_number	int
agency_name	string
last_name	string
first_name	string
middle_init	string
agency_start_date	date
work_location	string
title_description	string
leave_status	string
base_salary	int
pay_basis	string
regular_gross_paid	int
ot_hours	int
total_ot_paid	int
total_other_pay	int

Site Map



Task Breakdown

steven	PM, javascript/data visualization, css
alex	payroll_data setup, javascript
cindy	user_data setup/accounts functionality, css, javascript/data visualization
jeff	javascript/data visualization, css

Front-end Framework

Tailwind: because of our familiarity with tailwind, it is the most efficient for our purposes, some of which may include:

- Navbar → for map.html styling
- Cards → for register and login styling
- Drawers → for user customization in map.html

APIs

None

Data Visualization Library

PROS: d3.js	**CONS: d3.js**
<u>More versatility</u> d3 enables custom shapes (which we will need for the map), and chart.js does not.	<u>Higher effort</u> chart.js offers a good selection of built-in chart types, but selecting d3 for custom chart types means putting in more effort.
<u>More engaging visuals</u> d3 also has higher potential for interactive elements, which our project emphasizes.	<u>Steep Learning Curve</u> d3 is definitely more complicated to wield well than chart.js.

Because the highlight (and purpose) of our project is the data visualizations, **our team elected to use d3.js** because it supports our vision. On its website are several examples that we might want to implement in our map of NYC. For instance, they have a map of the US with regional coloring based on obesity, as well as another one where you can zoom into a US state when clicked on.